

Engineer On Call (EOC) for GitLab.com

To start with the right foot let's define a set of tasks that are nice things to do before you go any further in your week.

By performing these tasks we will keep the broken window effect under control, preventing future pain and mess.

Before your week of on-call shifts

- Join the `#eoc-general` slack channel
- Familiarize yourself with Pagerduty and Incident.io
- Make sure you can receive test pages from Pagerduty
- Be prepared to respond to alerts within 15 minutes during your on-call time

Beginning your daily on-call shift

Here is a suggested checklist of things to do at the start of an on-call shift:

- *Change Slack Icon:* Click name. Click **Set status**. Click grey smile face. Type `:pagerduty:`. Set **Clear after** to end of on-call shift. Click **Save**
- *Join alert slack channels if not already a member:*
 - `#production`
 - `#incidents-dotcom` (Declared incidents will appear in this channel)
 - `#incidents-dotcom-triage` (Triage incidents from alerts will appear in this channel)
 - `#alerts-prod-abuse`
 - `#g_security_vulnmgmt_notifications`
- *Turn on slack channel notifications for these slack channels for All new messages:*
 - `#incidents-dotcom`
 - `#incidents-dotcom-triage`
- At the start of each on-call day, read the on-call handover issue that has been assigned to you by the previous EOC, and familiarize yourself with any ongoing incidents.
- Review the active incidents and see if you need to take on the incident lead role, or if there are any outstanding actions.

Ending your daily on-call shift

- *Turn off slack channel notifications:* Open notification preferences in monitored Slack channels from the previous checklist and return alerts to the desired values.
- *Leave noisy alert channels:* `/leave` alert channels (It's good to stay in `#alerts` and `#alerts-general`)

- Make sure any high severity (S1/S2) incidents have been handed over to someone. If it's unclear who, ask the next on-call engineer to pick up the incident lead role in incident.io for those incidents.
- Update the on-call handover issue for the next EoC to include any important context or time sensitive tasks. For sensitive information, comment on the issue as in internal note.

Completing your week of on-call shifts

- Take a deep breath! You did it!
- Review your incidents and see if any of them need corrective actions, to be marked as resolved, or reviews filled out.
- Take note of any alerts that were not productive and use these resources to make notifications more helpful.
- Go to Workday and schedule a day off, selecting **Time Off in Lieu** as the type of absence. Being on call is stressful, even on a good week.

Things to keep an eye on

On-call issues

First check active GitLab.com production incident issues to familiarize yourself with what has been happening lately. Also, keep an eye on the #production and #incidents-dotcom channels for discussion around any on-going issues.

Useful Dashboard to keep open

- Platform Triage

Slack sre-oncall alias

In Slack, there is an alias (@sre-oncall) that will notify the current EoC. Responding to these requests is best effort and you are empowered to decide the best response given the current production situation. You can also always reach out to your team, the #eoc-general Slack channel, or others to help delegate work that is going to prevent you from being ready to respond to incidents.

Incidents

For each new incident you are aware of, regardless of the source, you should declare an incident in Incident.io. You can declare incidents in Slack using the /incident command. You can also declare incidents in the incident.io web application. We automatically create triage incidents based on paging alerts. These incidents are announced to #incidents-dotcom-triage. When you get a page that you need to turn into an incident, you can accept the associated triage incident to ensure that relevant alerts are linked to the incident.

Alerts

Alertmanager, deadmansnitch, incidents, and any other alert generating process are directed to the **GitLab Production** service in Pagerduty and will page someone (you). If you want to see current alerts, you can check Pagerduty. Alerts that page you or appear in Slack are not the only way to know there is an ongoing incident. You may be made aware of other problems from individuals mentioning problems to you, or opening incidents, or seeing problems first-hand. There is no wrong way to be made aware of an incident.

When you are paged, or are made aware of a problem, follow this general workflow of steps:

1. Acknowledge the alert if you can investigate and work the problem. You have 15 minutes to acknowledge before the alert is escalated automatically to the Incident Manager On Call (IMOC).
2. If the alert is not part of an existing incident, quickly assess the impact and open a new incident with an appropriate severity. If the severity is hard to determine, assume the impact is greater and open the incident with a higher severity.
3. During lower severity (S3/S4) incidents, you will be responsible for updating the incident overview.
4. During higher severity (S1/S2) incidents, the IMOC is usually tasked with making incident updates and helping coordinate resources in the incident.
5. Try to help log and note any characterization of the problem. Look for error logs, dashboards, etc, that help illustrate the cause, or symptoms of the incident. Look at feature categories and the service to help identify who might need to help solve this incident.
6. Be sure to look for recent deployments, or feature flags to see if they are related to the incident.
7. If you know what must be done to end the incident, perform the actions! If not, reach out to the most likely group who could help solve the incident, the IMOC, or others.
8. Once the incident is no longer impacting users, focus on the steps it may take to completely resolve the incident. Ideally, try to leave the environment in a stable state.
9. Identify follow-up actions (corrective actions or infradev issues) that would prevent the incident from re-occurring. Create those issues and link them as follow-ups in incident.io.
10. For higher severity incidents, help the IMOC get the incident review process in place, and update timelines, etc.

Security

If you find any abnormal or suspicious activity during the course of your on-call rotation, please do not hesitate to contact security.

Rotation Schedule

We use PagerDuty to manage our on-call rotation schedule and alerting for emergency issues. We currently have a split schedule between EMEA, AMER, and APAC for on-call rotations in each geographical region.

The AMER, APAC, and EMEA each have a shadow schedule which we use for on-boarding new engineers to the on-call rotations.

When a new engineer joins the team and is ready to start shadowing for an on-call rotation, overrides should be enabled for the relevant on-call hours during that rotation. Once they have completed shadowing and are comfortable/ready to be inserted into the primary rotations, update the membership list for the appropriate schedule to add the new team member.

This pagerduty forum post was referenced when setting up the blank shadow schedule and initial overrides for on-boarding new team members.

Creating temporary PagerDuty maintenance windows

A temporary maintenance window may be created at any time using the `/chatops gitlab run pager pause` command in the `#production` slack channel. The default window duration is 1 hour. To schedule a window for another duration a ruby `chronic`-compatible time specification can be used like so: `--duration="2 hours"`.

For more options, use `/chatops gitlab run pager --help`:

Pause or resume pages.

Usage: `pager <pause|resume> [options]`

Options:

<code>-h, --help</code>	Shows this help message
<code>--duration</code>	Duration of window; default: 1 hour
<code>--environment</code>	Environment [production,staging,test]; default: production
<code>--filter-by-creator</code>	Filter maintenance windows by creator; default: false

Currently a maintenance window cannot be created for a duration smaller than 1 minute, according to undocumented implementation in the PagerDuty API.